```
            await _emailSender.SendEmailAsync(to, subject, htmlMessage); ❼
        }                                                                ❼
    }
}
```

❶ A using directive is needed to bring the IEmailSender into scope.

❷ Implement the ILogger interface.

❸ Inject an IEmailSender and assign it to a field.

❹ This logger will not support scopes.

❺ If the log level is Error or Critical, this logger is enabled.

❻ Within the Log method, you establish whether the logger is enabled. If not, you return. Otherwise, you construct some HTML from the state and any exception that is passed in.

❼ The SendLog method uses the IEmailSender to send the email.

The `Log` method takes a number of parameters. The first is the `LogLevel` of the log. You will use this to determine if the logger is enabled. If it is, you ignore the `EventId` because you are not interested in it. Instead, you are interested in the `state` that contains any values passed in to the log, and you are interested in the `exception`, if there is one. The state can be cast to an `IEnumerable<KeyValuePair<string, object>>`, with each entry containing the name of a placeholder as its key and an object representing its value. When the framework logs an exception, it adds one entry to the collection, which has a key of `{OriginalFormat}` and a value of `An unhandled exception has occurred while executing the request`. You can get this value simply by calling `ToString()` on the state variable. Then you attempt to reference the message and stack trace of the exception and include those in the email message.

The next component you need to create is an implementation of `ILoggerProvider`, which is responsible for creating the logger on demand. Add another class file to the Logging folder named